

Технологии больших идей



95.79

Общий рейтинг

ИТ-холдинг Т1

Многопрофильный ИТ-холдинг

[Подписаться](#)



DKolesnikov

24 мая 2024 в 12:14

Основы безопасности в Kubernetes



Средний



15 мин



6.4K

Блог компании ИТ-холдинг Т1, Информационная безопасность*, Kubernetes*, DevOps*, Системное администрирование*

[Тutorial](#)



В моей роли DevOps-инженера, специализирующегося на Kubernetes, я регулярно сталкиваюсь с задачами, требующими глубокого понимания множества аспектов этой технологии. Особое внимание уделяется безопасности — критически важному условию для эффективного функционирования и защиты приложений в Kubernetes. Хотя безопасность часто может оставаться вне поля зрения при рассмотрении других операционных задач, её роль в успешном развёртывании и поддержке приложений нельзя недооценивать. Мой опыт и знания в этой области легли в основу данной статьи.

Мы сосредоточимся на двух ключевых элементах безопасности в Kubernetes: Role-Based Access Control (RBAC) и Pod Security Admission. Эти механизмы играют важную роль не только в

обеспечении безопасности приложений и данных в кластере, но и в управлении доступом и сетевыми взаимодействиями.

Элементы безопасности, такие как RBAC и Pod Security Admission, играют ключевую роль в обеспечении стабильности и эффективности работы приложений в Kubernetes, особенно при обработке больших объёмов данных и высокой нагрузке. Например, в ситуации с масштабируемым веб-приложением, которое управляет значительными объёмами пользовательских данных и транзакций, настройка этих механизмов может существенно улучшить управление доступом и сетевую безопасность. Это, в свою очередь, помогает предотвратить потенциальные угрозы и атаки, что критически важно для поддержания производительности и доступности данных. Таким образом, эффективно настроенные компоненты безопасности обеспечивают надёжный доступ к данным и минимизируют риски, связанные с увеличением нагрузки на приложение, улучшая общий пользовательский опыт.

Теперь давайте рассмотрим каждый из этих элементов более подробно.

Что же такое Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) в Kubernetes представляет собой мощный механизм для управления доступом к API Kubernetes. Он позволяет администраторам точно определять, кто (какие пользователи или процессы) и что может делать в кластере. Это достигается присвоением ролей и кластерных ролей, которые связываются с политиками доступа.

Основные Концепции RBAC

Роли и кластерные роли

Role (роль) в Kubernetes определяет набор разрешений в рамках конкретного пространства имён. Она может быть создана для предоставления доступа к чтению информации о подах только в определённом пространстве имён. Это ограничивает область действия роли и предотвращает её применение в других частях кластера.

ClusterRole (кластерная роль) похожа на роль, но её область действия распространяется на весь кластер. Она может использоваться для предоставления доступа к чтению информации обо всех узлах кластера или для управления ресурсами, которые не привязаны к конкретным пространствам имён, например, Custom Resource Definitions (CRD).

Разрешения

Verbs (действия) в контексте RBAC определяют действия, которые могут быть выполнены над ресурсами. Эти действия включают в себя стандартные операции CRUD (создание, чтение, обновление, удаление) и другие:

- **get** позволяет получать информацию о конкретном экземпляре ресурса. Это основная операция для чтения подробностей об объекте, таких как конфигурация пода или статус узла.
- **list** позволяет перечислять все экземпляры ресурса в пространстве имён или кластере. Это важно для сценариев, где требуется обзор или аудит ресурсов.
- **create** разрешает создание нового экземпляра ресурса. Это критически важно для операций развёртывания и управления жизненным циклом приложений.
- **delete** позволяет удалять ресурсы. Эта операция необходима для управления жизненным циклом ресурсов и очистки неиспользуемых или устаревших объектов.
- **update** авторизует обновление существующего ресурса. Это часто используется для изменения конфигурации ресурсов, таких как обновление версии образа в поде.
- **patch** похож на update, но позволяет вносить в ресурс частичные изменения. Это особенно полезно для динамических обновлений, когда нужно изменить только определённые поля объекта без его отправки целиком.

- **watch** позволяет отслеживать изменения ресурсов в реальном времени. Это важно для систем мониторинга и автоматизации, которым требуется немедленно реагировать на изменения в кластере.

Resources (ресурсы) указывают на объекты Kubernetes, к которым применяются разрешения. Это могут быть поды, службы, узлы и другие:

- **Pods** (поды): управление доступом к подам позволяет контролировать, кто может создавать, просматривать, обновлять или удалять поды в кластере.
- **Services** (сервисы): управление доступом к сервисам важно для контроля трафика и доступности приложений.
- **ConfigMaps** и **Secrets** контроль доступа к этим ресурсам критически важен для поддержания безопасности и целостности конфигураций приложений и инфраструктуры.
- **Deployments** и **StatefulSets** управление доступом к этим ресурсам позволяет контролировать, кто может изменять состояние приложений в кластере.
- **Nodes** (узлы): доступ к узлам позволяет управлять их настройками, мониторингом и обслуживанием.

API Groups: Kubernetes API разбит на группы для логического разделения ресурсов и функциональности. Каждая группа содержит связанные через API ресурсы и операции:

- **Core** (основная группа) содержит основные ресурсы, такие как поды, сервисы и узлы. Эта группа не имеет явного имени группы в пути API и часто обозначается как "" (пустая строка).
- **Apps** включает в себя ресурсы, связанные с приложениями, такие как Deployments, StatefulSets и DaemonSets. Эта группа помогает управлять жизненным циклом приложений в кластере.
- **Batch** содержит ресурсы для пакетной обработки, в том числе Job и CronJob. Эта группа используется для задач, которые выполняются один раз или по расписанию.
- **Extensions** и **Networking** включают в себя дополнительные ресурсы, такие как Ingress, которые расширяют функциональность Kubernetes в области сетевых сервисов и конфигураций.
- **Custom Resource Definitions (CRD)** позволяют пользователям создавать свои собственные ресурсы, расширяя возможности Kubernetes. CRD важны для интеграции сторонних решений и создания операторов Kubernetes.

Примеры политик:

Только чтение для подов в пространстве имён «test»:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: test
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

Эта политика позволяет пользователям получать, перечислять и отслеживать поды только в пространстве имен test .

Управление сервисами на уровне кластера:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: service-manager
rules:
- apiGroups: [""]
  resources: ["services"]
  verbs: ["get", "list", "create", "update", "delete"]
```

Эта политика предоставляет полный доступ к управлению сервисами во всём кластере, позволяя создавать, обновлять и удалять сервисы.

Управление конфигурациями и секретами в пространстве имён «prod»:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: prod
  name: config-secret-manager
rules:
- apiGroups: [""]
  resources: ["configmaps", "secrets"]
  verbs: ["get", "list", "create", "update", "delete"]
```

Привязки ролей

RoleBinding позволяет назначать роли пользователям, группам или сервисным аккаунтам в рамках определённого пространства имён. Это обеспечивает гранулярный контроль доступа к ресурсам и операциям внутри пространства имён.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
  name: "janedoe"
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

В этом примере RoleBinding назначает пользователю `janedoe` роль `pod-reader`, которая позволяет читать информацию о подах в пространстве имён `default`.



Время запустить свой стартап

роли пользователям, группам или сервисным аккаунтам, предоставляя доступ к ресурсам во всех пространствах имён.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: read-nodes-global
subjects:
- kind: User
  name: "johndoe"
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io
```

Здесь ClusterRoleBinding предоставляет пользователю johndoe глобальные права для чтения информации об узлах кластера с помощью роли node-reader .

Для понимания того, как работает RBAC в Kubernetes «под капотом», необходимо рассмотреть не только концепцию, но и внутреннюю реализацию и взаимодействие с Kubernetes API. Давайте немного углубимся в технические детали.

Аутентификация и авторизация

Аутентификация: прежде чем применится RBAC, Kubernetes должен аутентифицировать пользователя или процесс. Это может быть выполнено через сертификаты, токены сервисных аккаунтов или другие средства аутентификации.

Авторизация: после аутентификации запрос пользователя или сервиса передаётся системе авторизации. Если используется RBAC, Kubernetes проверяет соответствующие привязки ролей и кластерных ролей.

Проверка разрешений

Когда поступает запрос к API, сервер обрабатывает его и определяет, требуется ли авторизация. Запрос сопоставляется с соответствующими ролями или кластерными ролями, чтобы увидеть, разрешены ли запрашиваемые действия (verbs) над ресурсами (resources). Если у роли или кластерной роли нет соответствующих прав, запрос отклоняется.

Взаимодействие RBAC с API

RBAC использует API группы rbac.authorization.k8s.io. Когда создаются или изменяются объекты RBAC (например, Role или RoleBinding), они сохраняются в etcd, как и другие объекты Kubernetes. При запросе к API контроллер RBAC на сервере API проверяет эти объекты, чтобы определить разрешения.

RBAC и Admission Controllers

Admission Controllers в Kubernetes — это плагины, которые действуют как защитные ворота, проверяющие запросы к серверу API после аутентификации и авторизации, но перед сохранением объекта в etcd. Они могут изменять или отклонять запросы.

Взаимодействие с RBAC

RBAC определяет, кто (какой пользователь или сервис) и какие действия может выполнять, основываясь на назначенных ролях и привилегиях. Это первый уровень проверки, который определяет, имеет ли субъект запроса право на выполнение операции.

После того как RBAC даёт «зелёный свет», запрос проходит через Admission Controllers, которые могут дополнительно ограничивать или модифицировать запросы на основе более сложных правил и политик. Эти контроллеры могут применять дополнительные ограничения, например

запрещать использование определённых образов контейнеров или налагать ограничения на ресурсы.

Примеры Admission Controllers:

- **PodSecurityPolicy** позволяет определять, какие поды могут быть созданы в кластере на основе уровня безопасности. Например, политика может запрещать создание привилегированных подов, ограничивать использование определённых возможностей контейнера или требовать, чтобы все поды работали под определённым UID.
- **ResourceQuota** работает вместе с RBAC для обеспечения соблюдения ограничений на использование ресурсов в пространстве имён. Это может включать в себя ограничения на общее количество используемых ресурсов, таких как CPU и память, а также ограничения на количество объектов, таких как поды, сервисы или персистентные тома.

Кеширование RBAC

Правила RBAC кешируются в Kubernetes для улучшения производительности сервера API. Это позволяет уменьшить задержку при частых запросах на авторизацию, поскольку серверу не требуется каждый раз обращаться к etcd для получения правил RBAC. Такой подход значительно снижает нагрузку на etcd и уменьшает общее время отклика системы, особенно в крупных кластерах с интенсивным использованием RBAC.

Управление изменениями политик и кешированием

Изменения в политиках RBAC могут не применяться мгновенно из-за механизмов кеширования. Это приводит к временным несоответствиям в доступе, когда новые политики ещё не вступили в силу. Для управления этими задержками:

- планируйте изменения вне пиковых часов и учитывайте потенциальную задержку в применении политик;
- используйте инструменты, такие как `kubectl rollout restart`, для сервисов и развёртываний, чтобы принудительно обновить политики безопасности. Также можно использовать инструменты мониторинга и оповещения для отслеживания статуса применения политик RBAC.

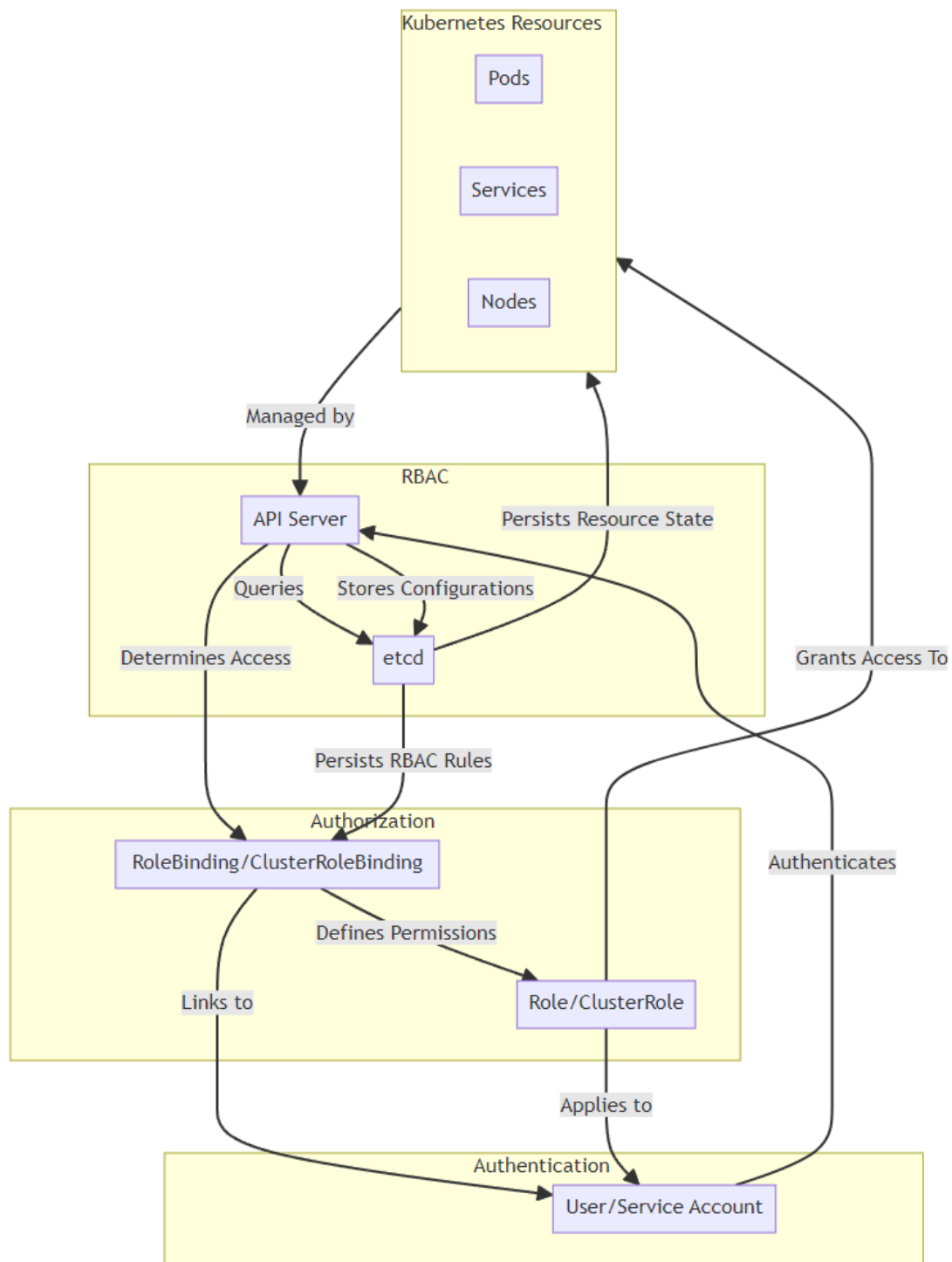
RBAC и расширения API

Custom Resource Definitions (CRD) предоставляют мощный инструмент для расширения функциональности Kubernetes, позволяя пользователям добавлять свои собственные ресурсы. Эти пользовательские ресурсы интегрируются с Kubernetes API и могут использоваться аналогично встроенным ресурсам, таким как Pods, Deployments и Services.

CRD могут использоваться для добавления новых функций в ваш кластер, таких как специализированные базы данных, конфигурации приложений или политики безопасности. Также CRD позволяют интегрировать внешние системы и сервисы с вашим кластером Kubernetes, обеспечивая единый интерфейс управления.

RBAC позволяет точно управлять доступом к пользовательским ресурсам, определённым через CRD. Вы можете создавать роли и привязки ролей, которые определяют, кто может создавать, просматривать, изменять или удалять экземпляры ваших CRD.

Схема внутренней работы RBAC (Role-Based Access Control) в Kubernetes:



На схеме изображены следующие компоненты и взаимодействия:

- API Server (сервер API): принимает запросы на доступ к Kubernetes API.
- etcd (хранилище ключ-значение): отвечает за хранение конфигурации кластера Kubernetes, включая правила RBAC.
- Authentication (аутентификация): в этой части проверяется подлинность пользователей или сервисных аккаунтов.
- Authorization (авторизация): проверяются разрешения доступа, определённых в RBAC.
- RoleBinding/ClusterRoleBinding (привязки ролей и кластерных ролей): эти объекты связывают пользователей или сервисные аккаунты с ролями или кластерными ролями.
- Role/ClusterRole (роли и кластерные роли): определяют набор разрешений, доступных пользователям или сервисным аккаунтам.

- **Kubernetes Resources** (ресурсы Kubernetes): здесь представлены различные ресурсы в Kubernetes, такие как поды, сервисы и узлы.
- **Stores Configurations** (хранение конфигураций): это указывает, что сервер API хранит конфигурации RBAC и ресурсов в etcd.
- **Persists RBAC Rules** (хранение правил RBAC): этот блок отображает, как правила RBAC сохраняются в etcd.
- **Persists Resource State** (хранение состояния ресурсов): это указывает на то, что состояние ресурсов также хранится в etcd.
- **Links to** (Связь с): показывает связь между пользователями или сервисными аккаунтами и правилами RBAC.

Лучшие практики

- **Принцип наименьших привилегий (Principle of Least Privilege)** Назначайте пользователям и сервисным аккаунтам только те разрешения, которые им действительно необходимы. Минимизируйте использование Wildcards: избегайте использования широких разрешений, таких как * для действий или ресурсов.
- **Минимизация использования ClusterRoles и ClusterRoleBindings** Использование Namespace-Scoped и Cluster-Scoped Roles: в зависимости от того, нужен ли доступ на уровне пространства имен или на уровне всего кластера, выбирайте Role или ClusterRole. Для большинства пользователей и сервисных аккаунтов предпочтительнее использовать области пространства имён (Namespaces) и связанные с ними роли (Roles).
- **Регулярный аудит и обновление правил RBAC** Периодически проверяйте и обновляйте свои правила RBAC, чтобы они соответствовали изменяющимся требованиям безопасности и бизнеса. Убедитесь, что устаревшие и ненужные разрешения удаляются.
- **Используйте Service Accounts с осторожностью** Service Accounts могут использоваться для автоматической аутентификации приложений в Kubernetes. Назначайте им только необходимые права и устанавливайте ограничения на их использование.
- **Тестирование политик RBAC** Тестируйте новые политики RBAC перед внедрением, чтобы убедиться, что они работают ожидаемым образом. Используйте такие средства как `kubectl auth can-i`, чтобы проверить, может ли пользователь выполнить определённое действие.

Зачем нужны сетевые политики

Сетевые политики в Kubernetes не только обеспечивают безопасность и изоляцию подов, но и предоставляют гибкие инструменты для управления сетевым трафиком на основе меток и селекторов. Это позволяет администраторам кластера детально определять, какие поды могут общаться друг с другом, тем самым минимизируя риск внутренних и внешних угроз.

Основные функции сетевых политик:

- **Принцип «минимального доступа».** Это ограничение коммуникации между подами до минимально необходимого уровня. Помогает предотвратить распространение атак в случае компрометации одного из подов.
- **Динамическое управление трафиком.** В отличие от традиционных сетевых экранов, сетевые политики Kubernetes позволяют автоматически применять правила к подам на основе их меток, без необходимости вручную обновлять правила для каждого нового пода.

Примеры применения:

- **Микросегментация** особенно полезна в многотенантных средах и приложениях с микросервисной архитектурой, где необходимо строго контролировать взаимодействие между различными компонентами системы.

- Ограничение доступа к подам из внешних сетей, с разрешением к внешним интерфейсам только определённых видов трафика, таких как HTTP или HTTPS.
- Контроль межсервисного взаимодействия. В микросервисных архитектурах часто требуется ограничить взаимодействие между сервисами, разрешая доступ только к необходимым компонентам. Сетевые политики позволяют точно контролировать, какие поды могут взаимодействовать друг с другом.
- Защита конфиденциальных данных. Сетевые политики помогают предоставлять доступ к конфиденциальным данным только для доверенных подов.

Примеры политик:

Default Deny All Ingress. Эта политика устанавливает строгий контроль, блокируя весь входящий трафик ко всем подам в определённом пространстве имён. Никакие внешние поды не смогут инициировать связь с подами в этом пространстве имён, пока не будут установлены более конкретные правила.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: example-namespace
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

Allow Specific Ingress. Позволяет входящий трафик к подам только от определённых источников, которые могут быть указаны через под-селекторы, пространства имён или даже IP-адреса. Это позволяет создавать более гранулированные правила доступа, обеспечивая доступ только доверенным источникам.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-specific-ingress
  namespace: example-namespace
spec:
  podSelector:
    matchLabels:
      app: myapp
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: trustedapp
        - namespaceSelector:
            matchLabels:
              name: trusted-namespace
```

Pod Security Admission в Kubernetes

Pod Security Admission (PSA) в Kubernetes — это механизм для замены устаревших Pod Security Policies (PSP). PSA предоставляет более простой и гибкий способ обеспечения безопасности подов в кластере. Он позволяет администраторам определять уровни безопасности для подов на основе предустановленных профилей безопасности.

Ключевые аспекты Pod Security Admission

- Профили безопасности:
 - **Privileged** предназначен для подов, которым требуются расширенные привилегии. Он идеально подходит для системных компонентов кластера, управляемых доверенными операторами, но его использование для обычных приложений не рекомендуется из-за повышенных рисков безопасности.
 - **Baseline** служит «золотой серединой», обеспечивая базовый уровень безопасности, который подходит для большинства приложений. Профиль предотвращает наиболее распространённые угрозы без значительного ограничения функциональности.
 - **Restricted** — строгий профиль, реализующий лучшие практики безопасности и минимизирующий поверхность атаки. Рекомендуется для приложений с высокими требованиями к безопасности.
- **Применение политик** Профили безопасности PSA применяются на уровне пространства имён с помощью аннотаций, что позволяет администраторам тонко настраивать политики безопасности в зависимости от требований к каждому пространству. Это обеспечивает гибкость в управлении и позволяет адаптировать политики к различным сценариям использования.
- **Совместимость с существующими подами** PSA учитывает необходимость совместимости с существующими подами, предлагая механизмы для постепенного внедрения строгих политик безопасности. Это позволяет организациям переходить на использование PSA без риска нарушения работы текущих приложений.

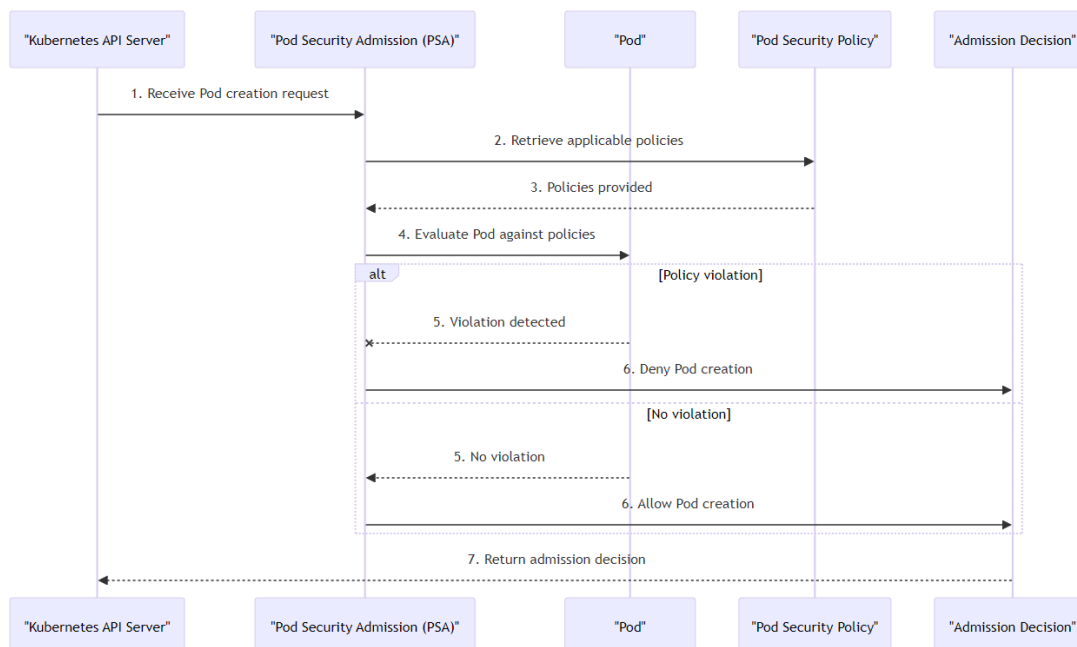
Практические примеры и рекомендации

Перед внедрением PSA в продуктивной среде рекомендуется провести аудит существующих подов и идентифицировать потенциальные изменения, необходимые для соответствия выбранным профилям безопасности.

Важно тестировать профили безопасности в тестовой среде, чтобы убедиться, что они не влияют на функциональность приложений и не приводят к нежелательным блокировкам трафика.

Аннотации пространства имён могут быть использованы для задания профилей безопасности по умолчанию и для указания допустимых профилей, что обеспечивает гибкость в управлении политиками безопасности на уровне пространства имён.

Схема, иллюстрирующая процесс Pod Security Admission (PSA) в Kubernetes:



Эта диаграмма представляет следующие шаги:

1. Сервер Kubernetes API получает запрос на создание пода.
2. PSA получает применимые политики безопасности пода.
3. Политики предоставляются PSA.
4. PSA оценивает под в соответствии с этими политиками.
5. Если обнаружено нарушение политики, PSA отказывает в создании пода. В противном случае создание пода разрешается.
6. Наконец, решение о допуске (разрешить или отказать) возвращается на сервер Kubernetes API.

Примеры использования

Настройка профиля для пространства имён. Администратор может настроить пространство имён так, чтобы все новые поды автоматически соответствовали уровню безопасности baseline. Это делается добавлением к пространству имён аннотации `pod-security.kubernetes.io/enforce: baseline`.

```

apiVersion: v1
kind: Namespace
metadata:
  name: example-namespace
  labels:
    pod-security.kubernetes.io/enforce: baseline
  
```

Исключения для отдельных подов. Если нужно запустить под с более высоким уровнем привилегий в пространстве имён с ограниченным уровнем безопасности, то можно использовать аннотации для исключения конкретного пода из общих правил.

```

apiVersion: v1
kind: Namespace
metadata:
  name: secure-namespace
  
```

```
labels:
  pod-security.kubernetes.io/enforce: restricted
```

Для пода, которому требуется исключение, можно добавить соответствующую аннотацию:

```
apiVersion: v1
kind: Pod
metadata:
  name: privileged-pod
  namespace: secure-namespace
  annotations:
    pod-security.kubernetes.io/enforce: privileged
spec:
  containers:
    - name: privileged-container
      image: some-image
```

Суммируя

Безопасность в Kubernetes — это комплексная задача, требующая постоянного внимания и обновления. Использование RBAC, Pod Security Policies и сертификатов является ключевым для обеспечения безопасности кластера Kubernetes. Регулярный аудит, обновление политик и практик, а также обучение команды — неотъемлемые элементы поддержания безопасности на высоком уровне.

Также рекомендую другие мои статьи про Kubernetes:

- Сетевые возможности и управление трафиком
- Работа с хранилищами для инженеров

Теги: rbac, Role-Based Access Control, Pod Security Admission

Хэбы: Блог компании ИТ-холдинг Т1, Информационная безопасность, Kubernetes, DevOps, Системное администрирование

◆ +14

🔖 81



💬 2

Редакторский дайджест



Присылаем лучшие статьи раз в месяц

Оставляя свою почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



ИТ-холдинг Т1
Многопрофильный ИТ-холдинг

Сайт



12

0

Карма Общий рейтинг

Дмитрий Колесников @DKolesnikov

DevOps инженер | Поток данных

Подписаться

